

1강 - Basic

#Computer System

Hardware : Physical elements 물리적인 요소들

Software : Programs; A finite set of computer instructions 컴퓨터 명령들의 유한한 집합

A computer System is Dynamic! 컴퓨터 시스템은 동적이다. >> 변한다!

소프트웨어를 바꿈으로서 여러 가지 일을 할 수 있다.

Instruction(명령) : 컴퓨터(마이크로프로세서)가 인식하고, 기능을 수행할 수 있는 기계어 명령어(의 집합)

$x+y=z$ 같은 것은 한 개의 instruction이 아니다.

Load Register, R1, y 같은 한 개의 명령

Machine language(기계어) : 0과 1로 이루어진 컴퓨터가 직접 이해할 수 있는 언어

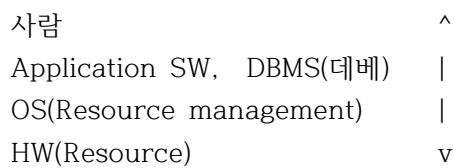
Assembly language : 어셈블리어. machine dependent. 프로세서 아키텍처, OS마다 다르다.

High level language : 사람이 이해할 수 있는 언어. C, C++, JAVA 등

Assembler : Assembly language -> Machine language

Compiler : High level language -> Machine language

#Hierarchy



OS : 시스템을 구성하기 위해서 반드시 있어야 할 SW == System Software

<2주차>

#Number System

Digit Position

10 Decimal $1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 = 123$

8 Octal $1 \times 8^2 + 2 \times 8^1 + 3 \times 8^0 = 83$

16 Hexa $1 \times 16^2 + 1 \times 16^1 + 1 \times 16^0 = 291$

A	B	C	D	E	F
10	11	12	13	14	15

2 Binary

Bit : Binary Digit; 0과 1을 저장할 수 있는 공간. 메모리의 최소 단위정보.

Byte : Bit 8개의 묶음 1Byte=8Bit

Kilo	Mega	Giga	Tera	Peta	Exa	Zeta	Yotta....
천	백만	십억	조	...			
2^{10}	2^{20}	2^{30}	2^{40}				
10^3	10^6	10^9	10^{12}				

Byte-addressable : 바이트 단위로 메모리 주소가 붙음. 바이트 단위로 접근 가능.

Word : 메인 메모리에서 CPU로 한 번에 읽어 들어올 수 있는 데이터의 단위.

32(64)bit machine : 한 번에 32bit 읽어올 수 있음.

CPU처리속도>>>데이터 읽어들이는 시간 이기 때문에 한 번에 많이 가져오면 처리 속도가 빨라진다.

Ex) 메모리가 1MB이면 주소가 몇 번지부터 몇 번지까지 있을까?

$10^6 = 2^{20}$ byte이므로 0번지~ $2^{20}-1$ 바이트까지

문자 하나는 1바이트에 저장 가능 >> 대표적 ASCII 코드

Ex) 1MB짜리 메모리의 주소를 표현하려면 최소 몇 bit가 필요한가?

1MB는 2^{20} byte이고 2^{20} 개의 주소를 표현해야 하므로 >> 20bit 필요. 3byte 할당해야 함.

#Number System conversion

(10) to (2)

(10) to (8) => 10진수를 (변환하려는 수)로 0이 될때까지 계속 나누고, 나머지를 역순으로 나열.

(10) to (16)

(2) to (8) => 3자리씩 묶어서 8진수로 표현

(2) to (16) => 4자리씩 묶어서 16진수로 표현

Algorithm == logic ; 문제를 해결하기 위한 일련의 명확하고 구체적인 단계적 절차

알고리즘의 5가지 특성

1. Input
2. Output
3. finiteness - 유한성; 반드시 끝나야 한다.
4. unambiguousness - 명확성; 애매모호하지 않아야 한다.
5. efficiency - 효율성; 효율적이어야 한다.

알고리즘을 표현하는 수단 : Natural Language(자연어), Pseudo Code(의사코드), flowchart

#Binary Arithmetic

Overflow: 오버플로우; 표현할 수 있는 데이터를 초과했을 때. > 컴퓨터는 finite하다.

$1011 + 1011 = 1/0110$ (22) >> Overflow돼서 22가 아니고 6으로 결과가 나온다.

<-> Underflow : 언더플로우; 유효숫자보다 초과하는 범위는 정확하지 않은 숫자가 저장되어 오차가 생김.

#Programming

Machine dependent : instruction set이 다르다! architecture도 다르다.

각 architecture는 특정한 instruction을 실행하기 위해서 하드웨어가 설계됨. 다른 Architecture에서는 작동x

instruction이 바뀌면 더 이상 작동하지 않음!

#instruction이 많으면 많을수록 좋을까?

A : 30개 - RISC

B : 50개 - CISC

instruction이 적으면 >>

같은 일을 더 적은 instruction으로 할 수 있다.

메모리에 2번 접근할 것을 1번에 할 수 있다.

but, 회로가 복잡해질 수 있다.

but, instruction의 길이가 길 수 있다.

>> 64비트가 한번에 더 긴 instruction을 가져올 수 있다.

2강 - Basic

#Clocks

: 컴퓨터 중앙에서 만들어지는 일련의 **전자적 펄스**
모든 컴퓨터 동작이 한 클럭마다 작동한다.
클럭이 1이면 - 입력이 걸리고, 0이면

clock pulse가 1일 때 입력이 들어오면 작동을 해서 출력이 나옴.
모든 일은 clock pulse가 1일 때(enable 됐을 때) 정상적으로 작동함.

clock이 높으면(주기가 더 짧으면) 같은 시간 안에 많은 일을 함.

2.66GHz : 초당 pulse가 26.6 bil번 생성됨.

MIPS : CPU의 빠르기를 측정하는 고전적인 방법 ; Million Instruction Per Second

#Memory

Volatile(휘발성) : **RAM**(Random Access Memory)
Non-volatile(비휘발성) : **ROM**(Read Only Memory), Flash Memory

Programmable ROM - 생산 후 사용자가 내용을 한 번만 기록할 수 있는 롬
EPROM - Erasable PROM; 지우고 쓸 수 있는 롬.
EEPROM - Elctircally EPROM : 전기적인 신호를 이용하여 지우고 쓸 수 있는 롬.

Firmware : 시스템을 기본 동작하는데 필요한 코드들

Byte-addressable : 바이트마다 주소가 붙어있음.

Register : CPU 안에 있는 임시 저장장치

RAM

DRAM : 주기적인 Refresh 필요	느림	쌈	집적도▲	전력소비▼
SRAM ; Refresh 불필요	빠름	비쌈	집적도▼	전력소비▲

Flash Memory, SSD(Solid State Drive) : ROM 형태의 특별한 메모리

#Cache

:Processor chip안에 built됨

	Cache	RAM	SSD
가격	비쌈	>>>>	쌈
속도	빠름	>>>>	비쌈
용량	작음	<<<<	큼

프로그램을 실행하려면 Main Memory에 로드가 되어있어야 함

Memory Management : 프로그램이 요청하면 메모리의 일부를 가져오고 더 이상 필요하지 않으면 할당 해제
>> 궁극적으로는 경제성 때문에 사용함.

“프로그램의 본질은 순차적 실행”. 그래서 그 다음에 오는 내용이 올 가능성이 높음

Reference(참조) : 특정 값에 접근하는 것

Locality of reference(참조의 지역성); 동일한 메모리 위치 집합에 접근하는 경향

Spatial Locality : 특정 저장 위치가 참조되면, 근접한 메모리 위치가 참조될 가능성이 높음.

Hit/Miss Ratio : CPU가 요청했을 때 해당 word가 cache memory에 있는 비율

Hit Ratio : $\text{Hit}/\text{Request}(\text{Hit}+\text{Miss})$

Hit Ratio $\propto$ 접근 속도 $\propto$ 실행 속도

Hit Ratio가 높을 수 있는 이유는? : Locality of reference 때문. 프로세서에서 요청한 word들이 인접해 있기 때문

RAM과 Cache의 Memory access 속도가 다르다.

Cache Coherence : Cash와 Memory에 있는 값이 같은 정도.

Write Through : CPU가 데이터를 Cache에 쓸 때 RAM에도 데이터를 쓰는방식

Write Back : Cache에만 쓰고, Cache에 있는 데이터가 제거될 때 주기억 장치에 데이터 씬.

새로운 데이터를 불러와야 하는데 어떤 곳에 불러와야(덮어써야) 할까?: Page Refreshment Strategy(Policy)

: LRU(Least Recently used:최근에 제일 덜 쓰인 것 우선)등등

#Cache Mapping

Address = tag+index

4비트 * 16크기의 RAM과 2비트 * 4바이트 Cache가 있음

어떤

주소는 보통 비트(bit) 단위로 나타내며, 이 비트들을 몇 개씩 끊어서 **태그**와 **인덱스**로 사용합니다.

태그(tag) : 캐시 메모리에 저장된 데이터가 **원래 주기억장치의 어느 위치에 있던 데이터인지**를 표시합니다. 캐시에서 데이터를 찾을 때, 요청된 주소의 태그와 캐시 내 데이터의 태그를 비교하여 같은 데이터인지 확인합니다.

인덱스(index) : 캐시 메모리 내에서 **데이터가 저장될 위치**를 결정합니다. 주소의 일부를 인덱스로 사용하여 캐시의 어떤 라인에 데이터가 저장될지를 결정합니다.

★캐시 메모리가 RAM에서 데이터를 불러오는 과정은 다음과 같습니다:

1. **주소 생성**: CPU가 데이터를 요청하면, 해당 데이터의 주소가 생성됩니다. 이 주소는 **태그**, **인덱스**로 나누어집니다.
2. **인덱스 확인**: 생성된 주소의 **인덱스** 부분을 확인하여, 해당 인덱스에 해당하는 **캐시 라인**을 찾습니다.
3. **태그 비교**: 찾아낸 캐시 라인의 **태그**와 주소의 **태그**를 비교합니다. 태그가 일치하면, 캐시 히트(cache hit)라고 하며, 해당 캐시 라인의 데이터를 CPU에 전달합니다.

4. **캐시 미스 처리**: 만약 태그가 일치하지 않으면, 캐시 미스(cache miss)라고 합니다. 이 경우, **해당 주소의 데이터를 RAM에서 가져와 캐시에 저장**하고, CPU에 전달합니다. 이 과정에서 캐시 교체 알고리즘(cache replacement algorithm)이 사용될 수 있습니다.

#Memory Hierarchy

Locality of reference : 참조지역성!

#Firmware

Low-level control(신호) program을 제공하는 소프트웨어의 일종. 시스템을 제어. 시스템의 기본적인 기능 수행.
@Non-volatile 비휘발성 메모리에 있음 - ROM 종류

Instruction을 수행하려면:

1. Cache에 접근해서 2진수로 된 명령어를 IR(Instruction register)로 가져옴. ex) AR R1 R2
2. instruction register에서 가져와서 해석..... 등등

>>**micro instruction**: instruction 하나를 수행 하기 위해 필요한 **각각의 하위 단계**

Micro code : 어떤 일을 실행하기 위한 0과 1로 이루어진(2진수로) 신호들의 집합.

Micro program :그 컴퓨터가 제공하는 모든 microcode의 집합.

Hard wired controled : 신호 발생을 하드웨어(물리적으로)로 함. 유연X

Microprogram controled : 신호 발생을 프로그래밍 한 것. 유연함.

원래는 computer's instruction set의 정의와 구현이 포함된 low-level microcode를 포함. (~신호를 저장)

그래서 원래는 ROM에 위치했었음(바꿀 일이 없어서)

현재는, 시스템의 기능이나 시스템을 저장하는 것으로 변함. Non-volatile memory(ROM, EPROM, flash memory)에 위치. 기능개선, 버그 수정 시 수정. BIOS, bootstrap loader(부트로더; OS를 찾아주는 SW), 특별한 어플리케이션(잘 바뀌지 않지만 특정한) 포함

대부분의 firmware는 업데이트 가능함.

#Booting

Power-on Self Test를 performing

Locating and initializing peripheral(주변) devices(I/O),
finding, loading and starting OS.

#BIOS

일종의 firmware

원래는 ROM-based firmware였다.

현재는 쉬운 업데이트를 지원. 새 기능 추가, 버그 수정 >> flash memory에 저장

BIOS rootkits로 바이러스가 감염될 수 있는 만들 가능성을 만들.

1. 하드웨어를 초기화(Hardware Initialization)

system hardware components를 초기화하고 테스트함.

POST : Power On / Self Test

Bootstrap Loader(Bootloader) : POST 이후 OS나 다른 시스템 SW를 로드하는 프로그램.

2. 입출력 장치 제공(I/O)

Basic Input, Output 기능의 작은 라이브러리를 제공.

키보드, 디스플레이, 다른 I/O 장치들

3. 런타임 서비스 제공(Runtime service provide)

실행 중에 Interrupt routines 상황 발생 시 OS나 프로그램이 필요한 필요한 서비스를 제공함.

#논리연산

NOT : $a = x'$

AND : $a = x \cdot y$

OR : $a = x + y$

XOR : $a = ab' + a'b$

NAND : $a = x' + y'$

NOR : $a = x'y'$